

How TanmayPortfolio completes the brief.

A React Native developer portfolio — built with Community CLI, run on Pixel_9, mapped 1:1 to Assignments 1 through 6.



Tanmay Shinde

B.Tech IT · MIT ADT · Full Stack & Software Developer

STACK

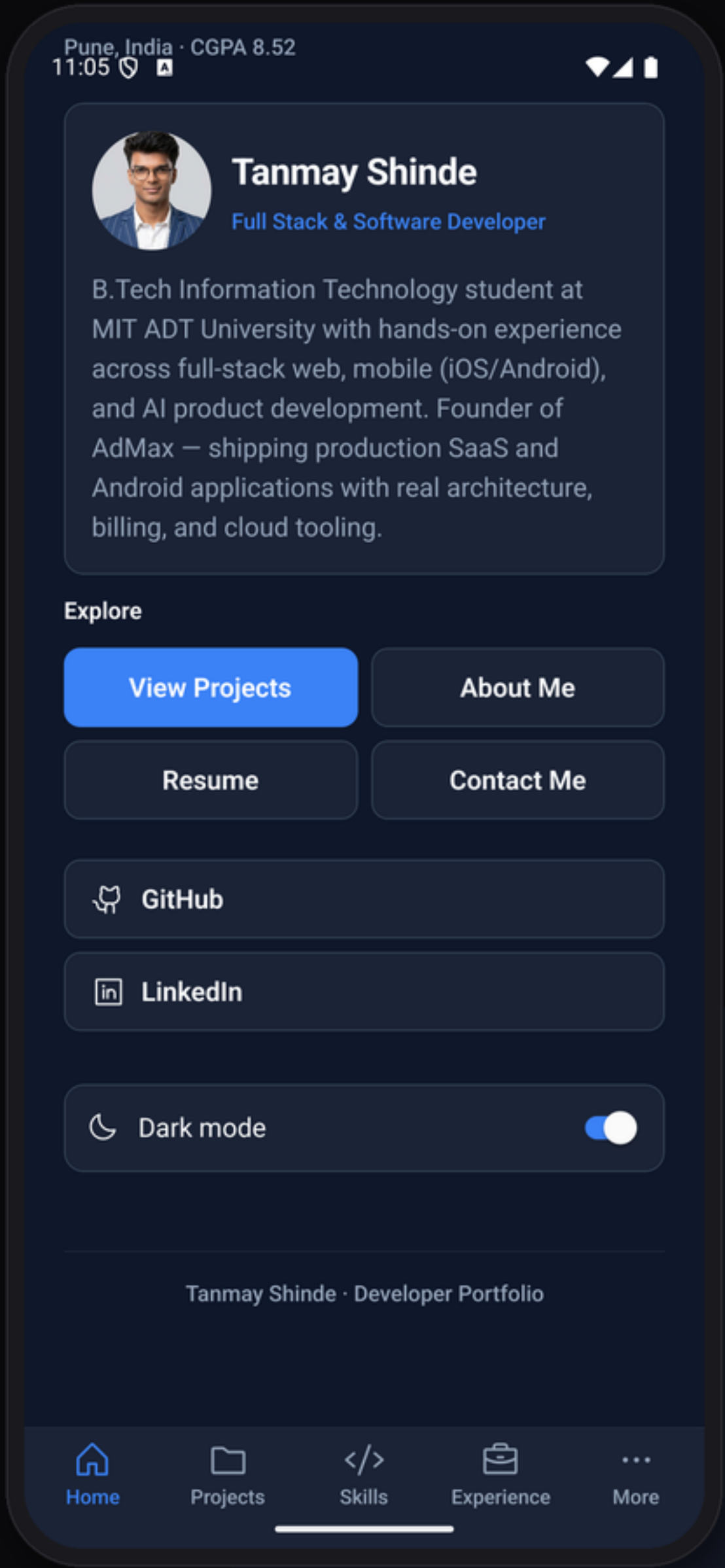
RN 0.87 · TypeScript

RUN

Pixel_9 emulator

TOOLING

CLI + Android Studio



Six assignments. One running app.

Each card is a viva checkpoint — requirement on the left, where to point in the project on the right.

01

Setup + Hello World

Community CLI · android/ · Pixel_9 · first emulator run

02

Comparison report

Native vs Hybrid vs React Native — and why RN here

03

Components, Props, useState

src/components · ProjectCard props · theme + form state

04

Folder structure + state

Documented src/ tree · Switch + validation updates

05

Profile app + core RN

View, Text, Image, TextInput, Button, Switch, ScrollView

06

Flexbox, TextInput, onPress

Skill wrap · contact inputs · Submit handler

Setup + Hello World

Evidence the project is a real Community CLI app that runs on the emulator.

Create RN project with Community CLI

Root TanmayPortfolio · package.json → react-native 0.87.1

android/ openable in Android Studio

Open the android/ Gradle project — not Expo Go

Run on emulator

npm run android / Studio Run on Pixel_9

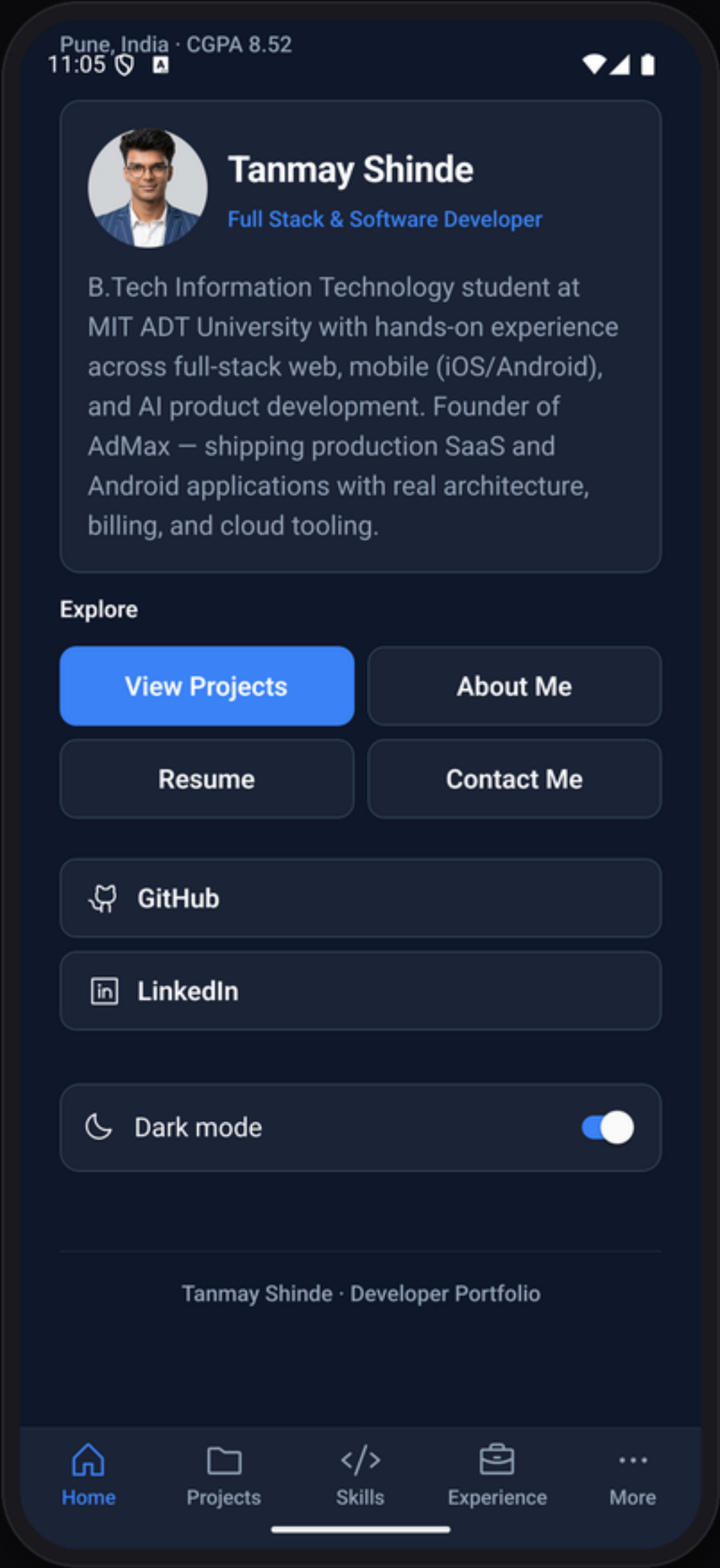
Hello World → running portfolio

Same CLI app; Home on Pixel_9 is the living first screen

Demo tip · CLI init → Metro → first successful emulator install

What to say

I used the React Native Community CLI so I get a real android/ folder. Metro bundles JS; Android Studio installs the APK on Pixel_9. The first milestone was a Hello World screen — the same app now hosts the full portfolio.



Pixel_9 · running portfolio Home

Native vs Hybrid vs React Native

Same product goal. Three ways to build. RN sits in the middle: shared JS, real native views.

Native

Approach

Kotlin / Swift + platform SDKs

Best performance & OS APIs

Two codebases to maintain

Banking, games, camera-heavy

Hybrid (WebView)

Approach

HTML / CSS / JS in a WebView

Fastest reuse of a website

Weaker gestures & native feel

News / brochure / simple forms

CHOSEN

React Native

Approach

TypeScript + React → native views

One UI codebase, Metro Fast Refresh

Need some Gradle / native tooling

This portfolio · product UIs · SaaS

Why React Native for this project



Curriculum fit

Functional components, Props, useState, core RN widgets, Flexbox — the assignment list is RN's native language.



Studio workflow

Community CLI ships android/. Open it in Android Studio, run Pixel_9, show Logcat — same as a native lab.



One codebase

A full student profile (Home → Contact) without maintaining separate Kotlin and Swift apps.



Industry signal

RN is widely used for cross-platform product UI — useful in internships, not only in the viva.

Submit: export docs/assignment-2-comparison-report.md to PDF if the college requires PDF. Native wins raw performance; Hybrid wins web reuse; RN wins this course.

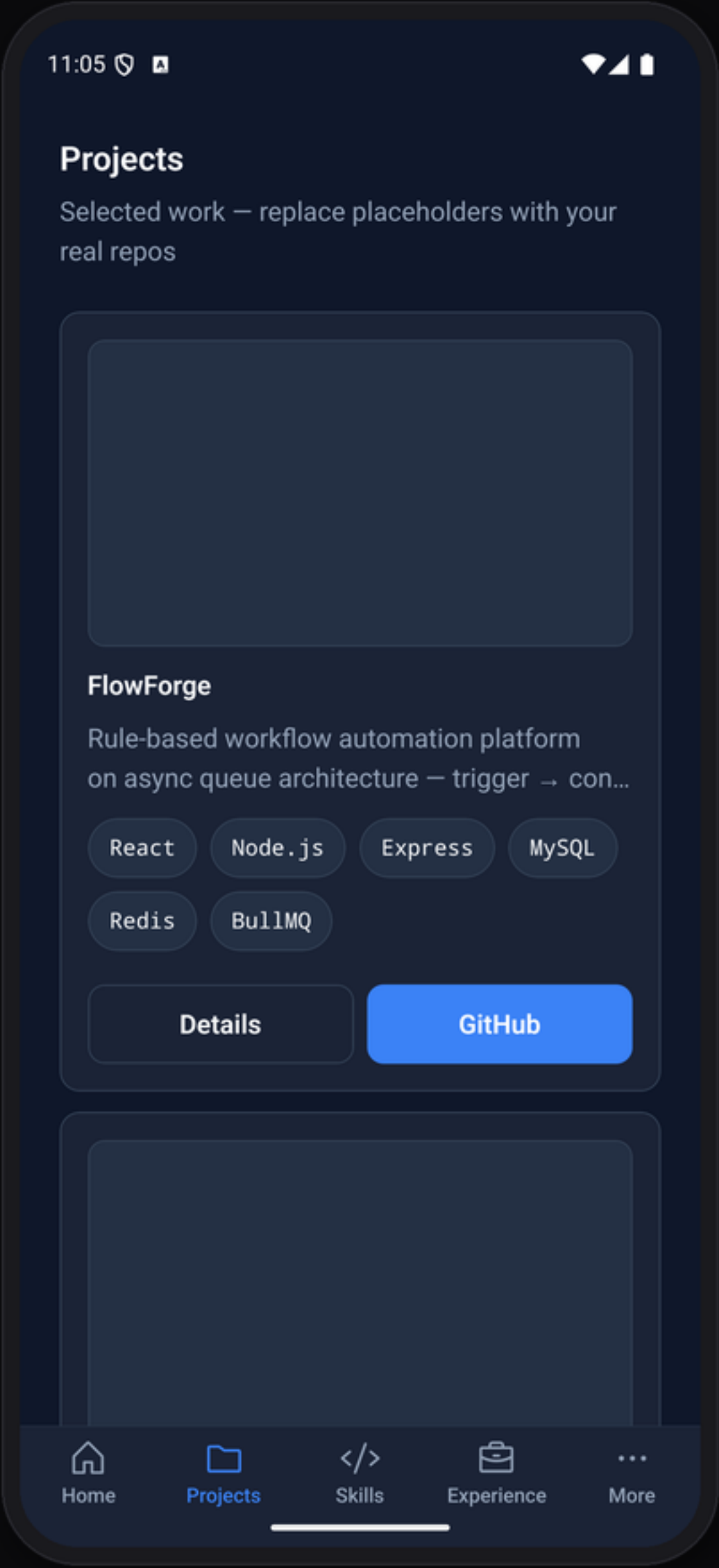
Reusable components + Props

Data lives in src/data. Components stay dumb and reusable.

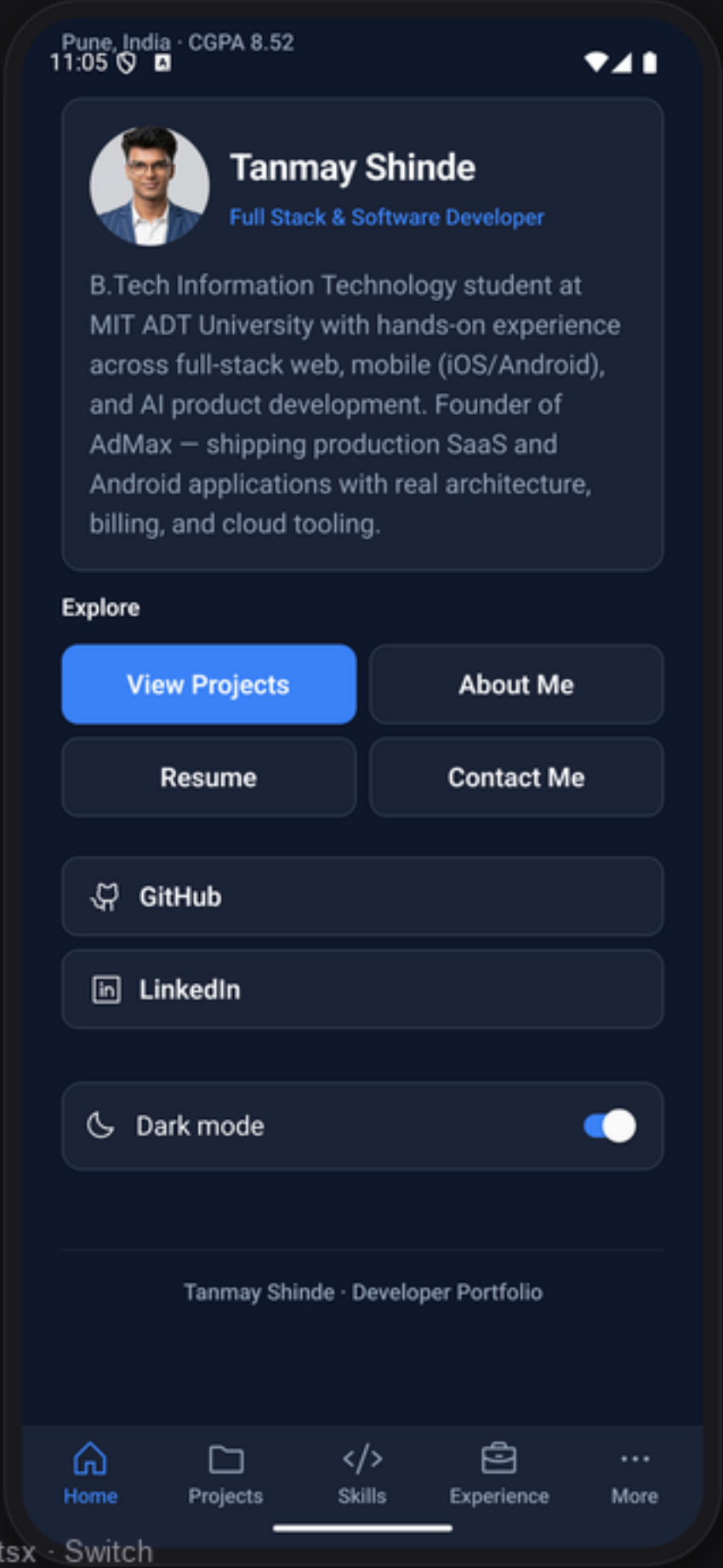
- ProjectCard**
title, description, tags via Props
- SkillChip**
label + accent — used in Skills & cards
- ThemeToggle**
controlled Switch, no hardcoded theme
- ContactForm**
stateful form composed of ContactInput

ProjectsScreen.tsx → data flow

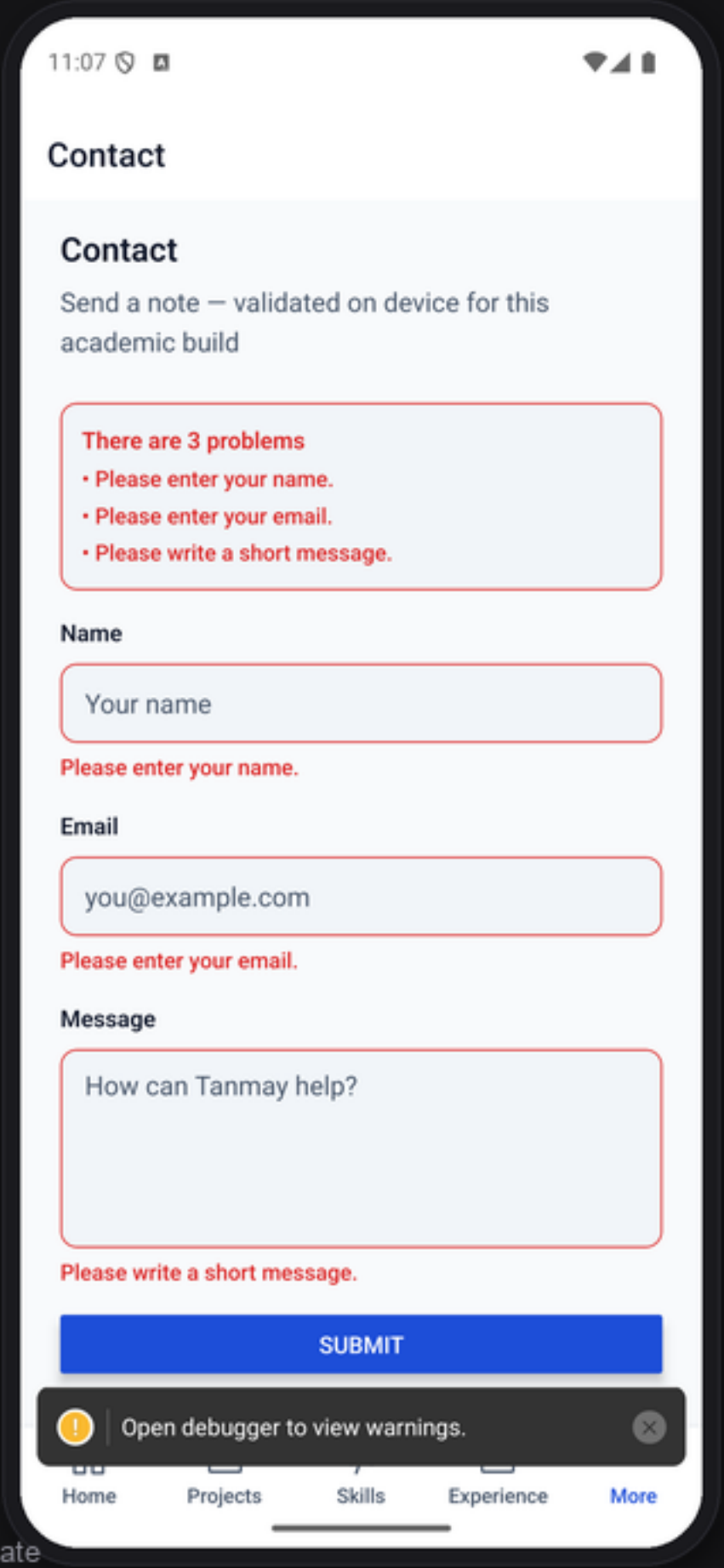
```
src/data/projects.ts ↓ map()
<ProjectCard title={...} tags={...} /> ↓ Details Pressable
expanded state lives inside the card (useState)
```



Dynamic UI from state — not from rewrites



Theme · isDark in App.tsx · Switch



Form · ContactForm errors · useState

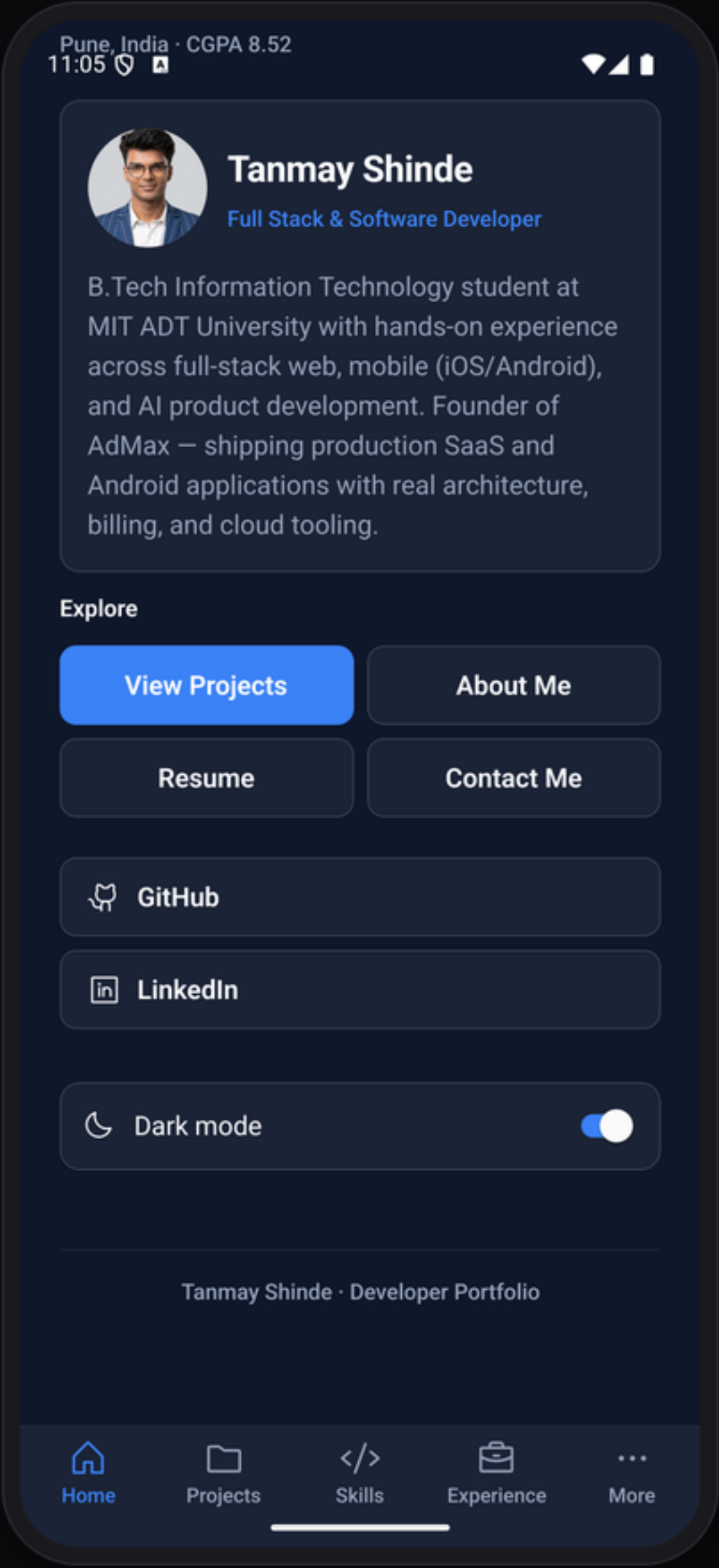
Folder structure that matches the concepts

android/ Native Gradle project — open in Android Studio	App.tsx Root useState(isDark) + ThemeProvider + Navigation
src/components/ Reusable UI: ProjectCard, SkillChip, ThemeToggle...	src/screens/ One screen per portfolio section
src/data/ Static content — keep components prop-driven	src/theme/ colors, tokens, ThemeContext
src/navigation/ Bottom tabs + More stack	docs/ Reports + screenshots for submission

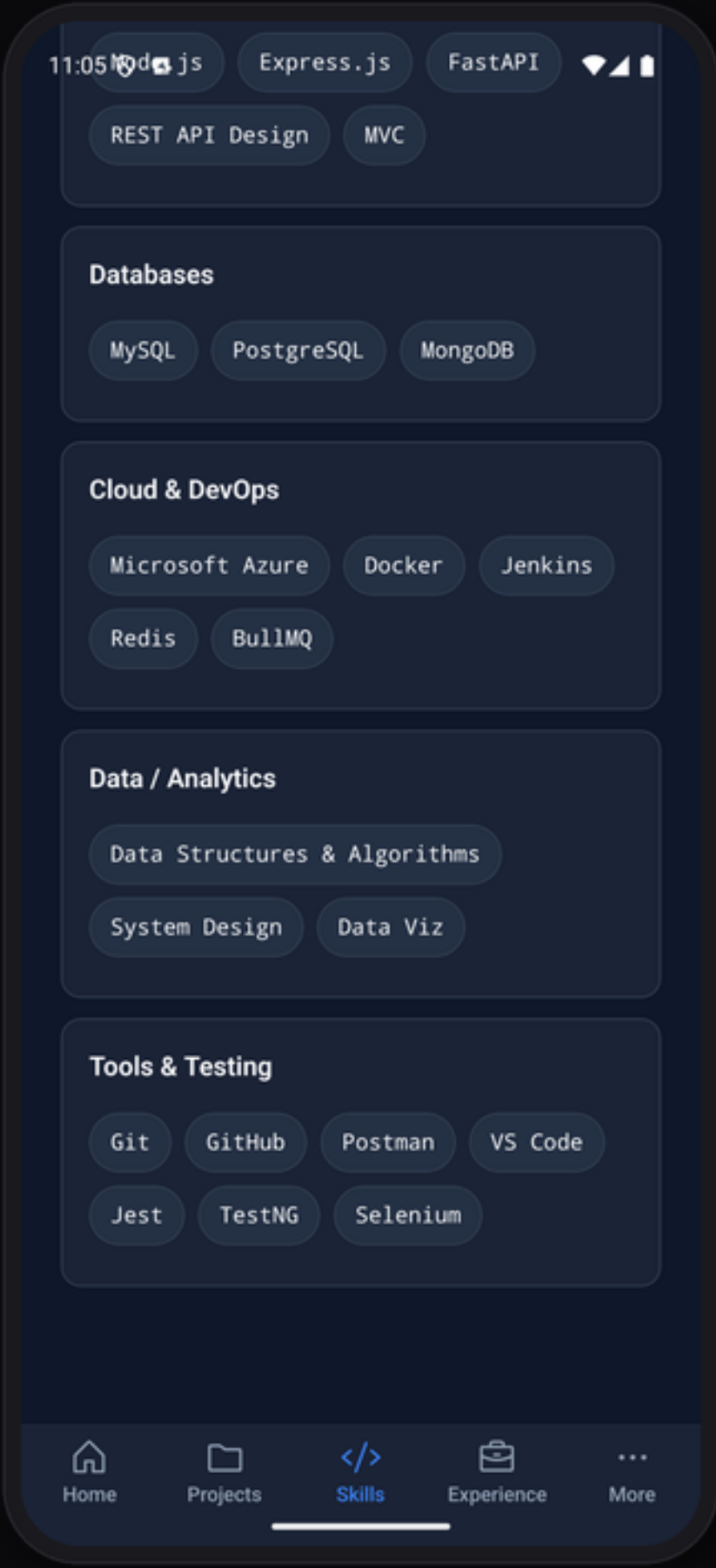
Student profile app — eight core components

Portfolio sections = a profile-style app: Home, About, Skills, Projects, Experience, Certs, Resume, Contact.

View Almost every layout	Text Titles, bio, errors
Image ProfileCard + profile.jpg	TextInput ContactInput fields
Button ContactForm submit	Switch ThemeToggle
ScrollView ScreenContainer	StyleSheet Every styled file



Every section is a profile screen



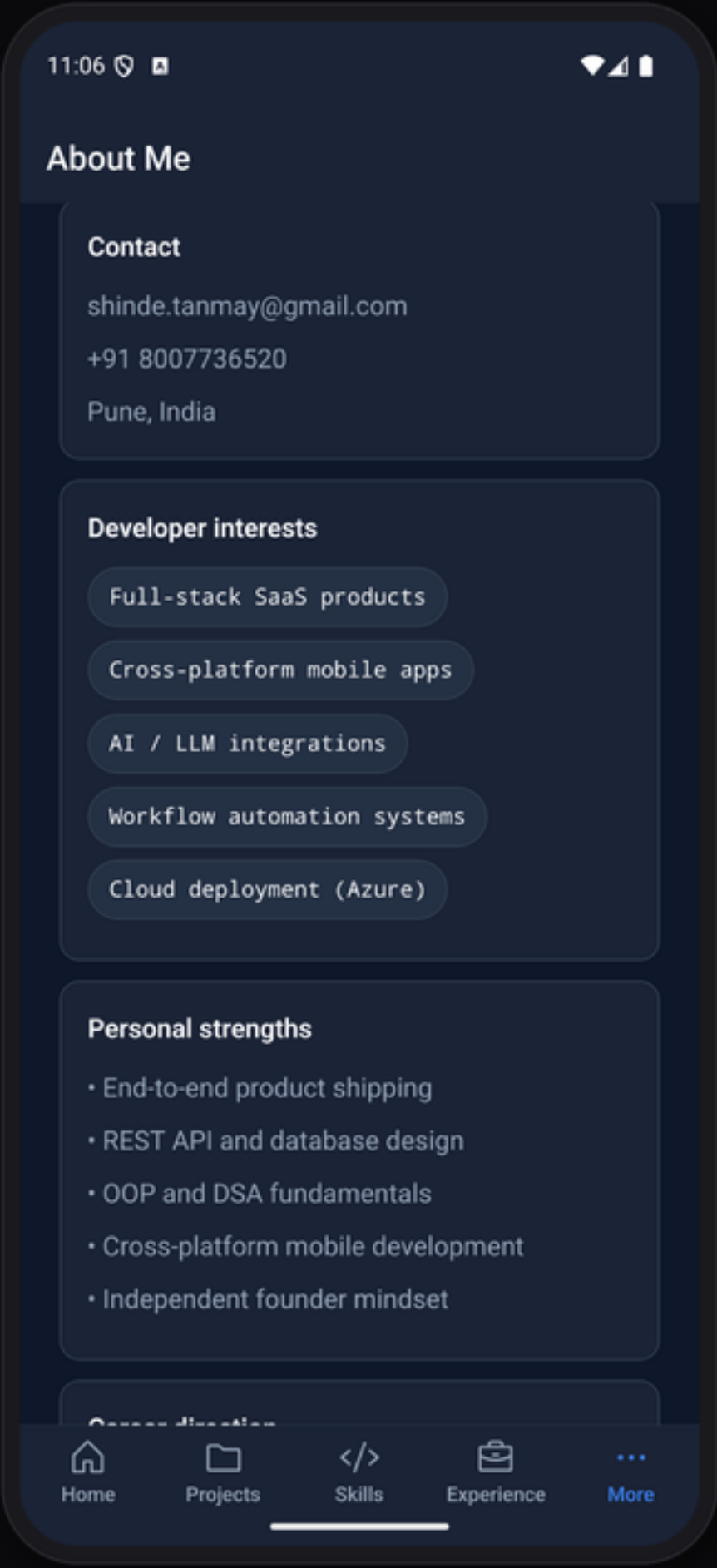
Skills

SkillChip + ScrollView



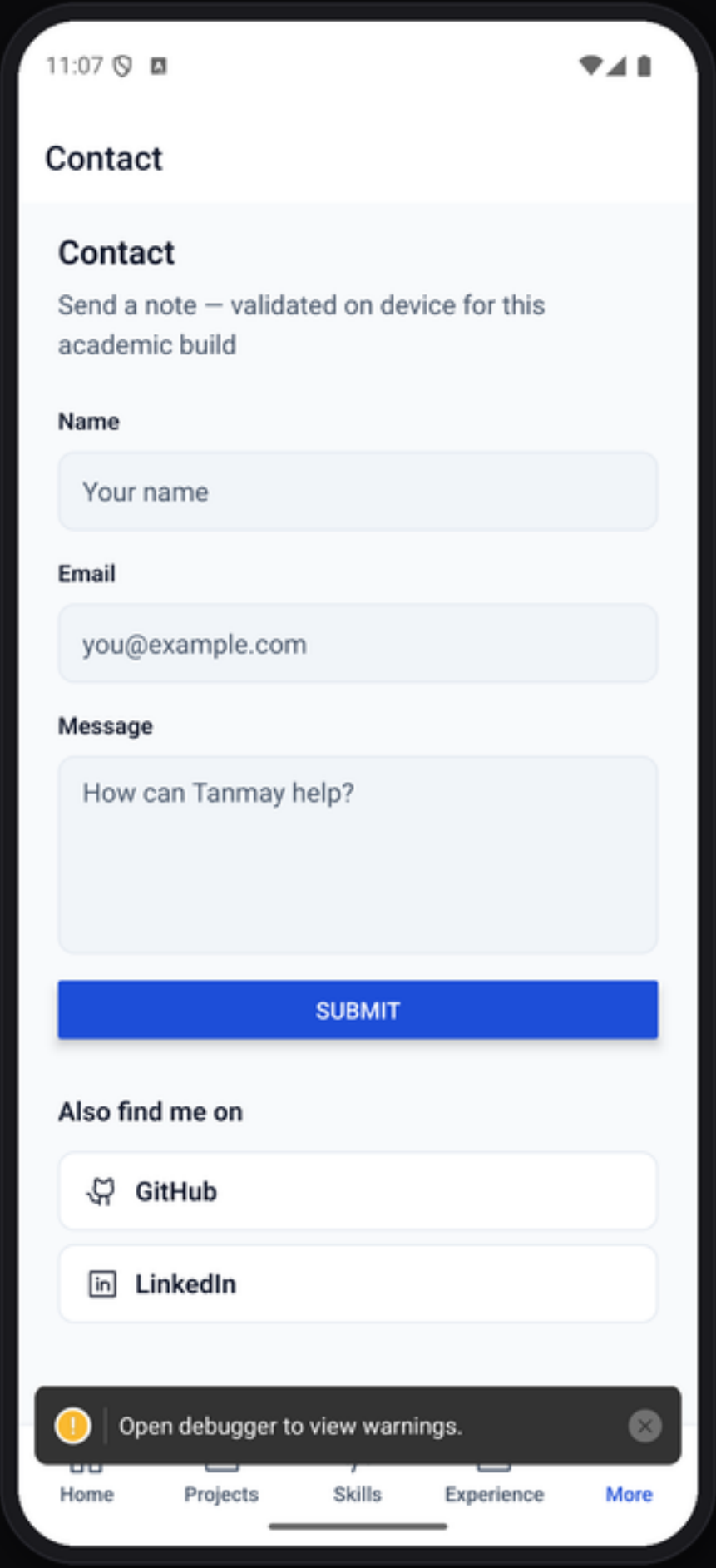
Experience

AdMax + CodSoft rail



More / About

Profile-style cards



Contact

TextInput + Button

Layouts that wrap, align, and rail

Home CTA wrap

Explore buttons use flexWrap so CTAs reflow instead of overflowing.

SkillCategory chips

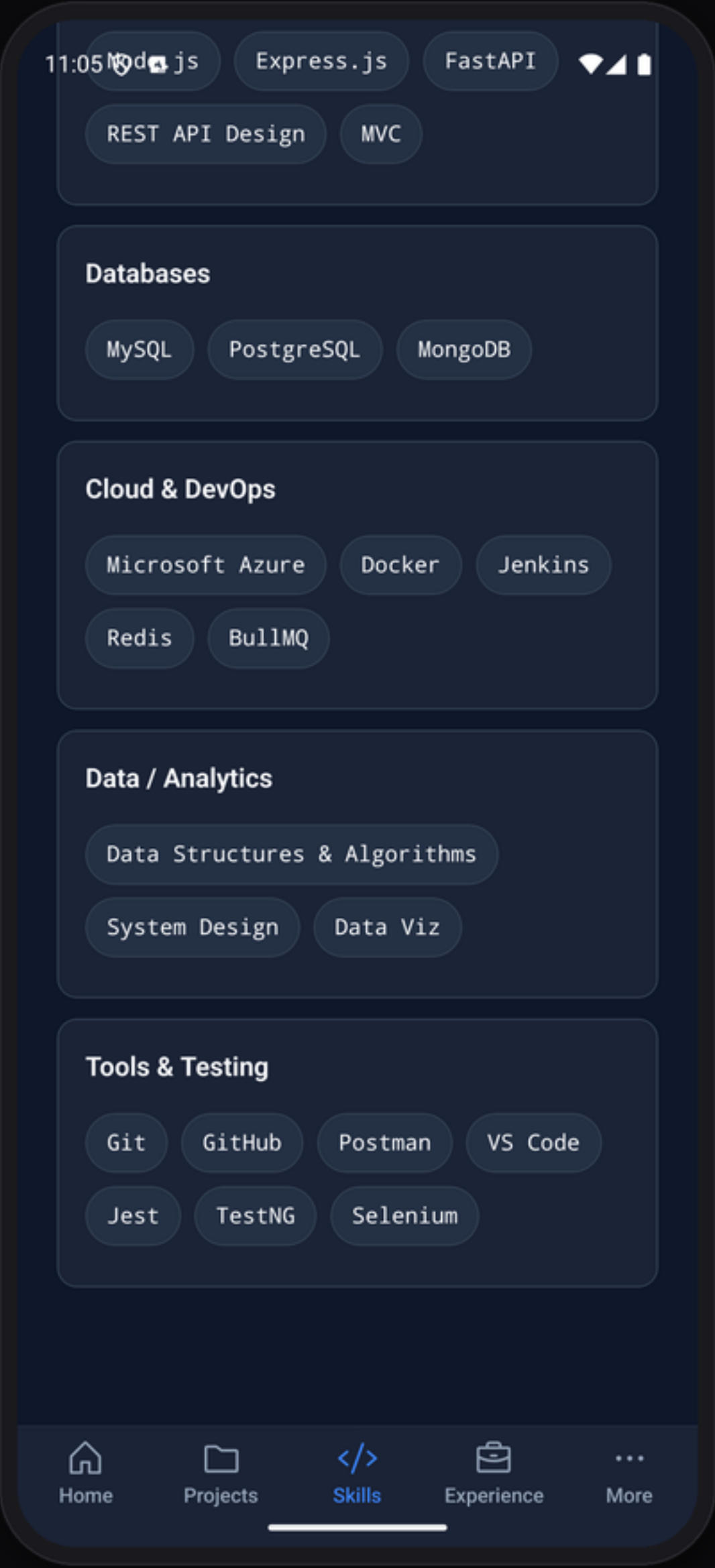
SkillChip row with flexWrap — tags sit on multiple lines.

Experience row + rail

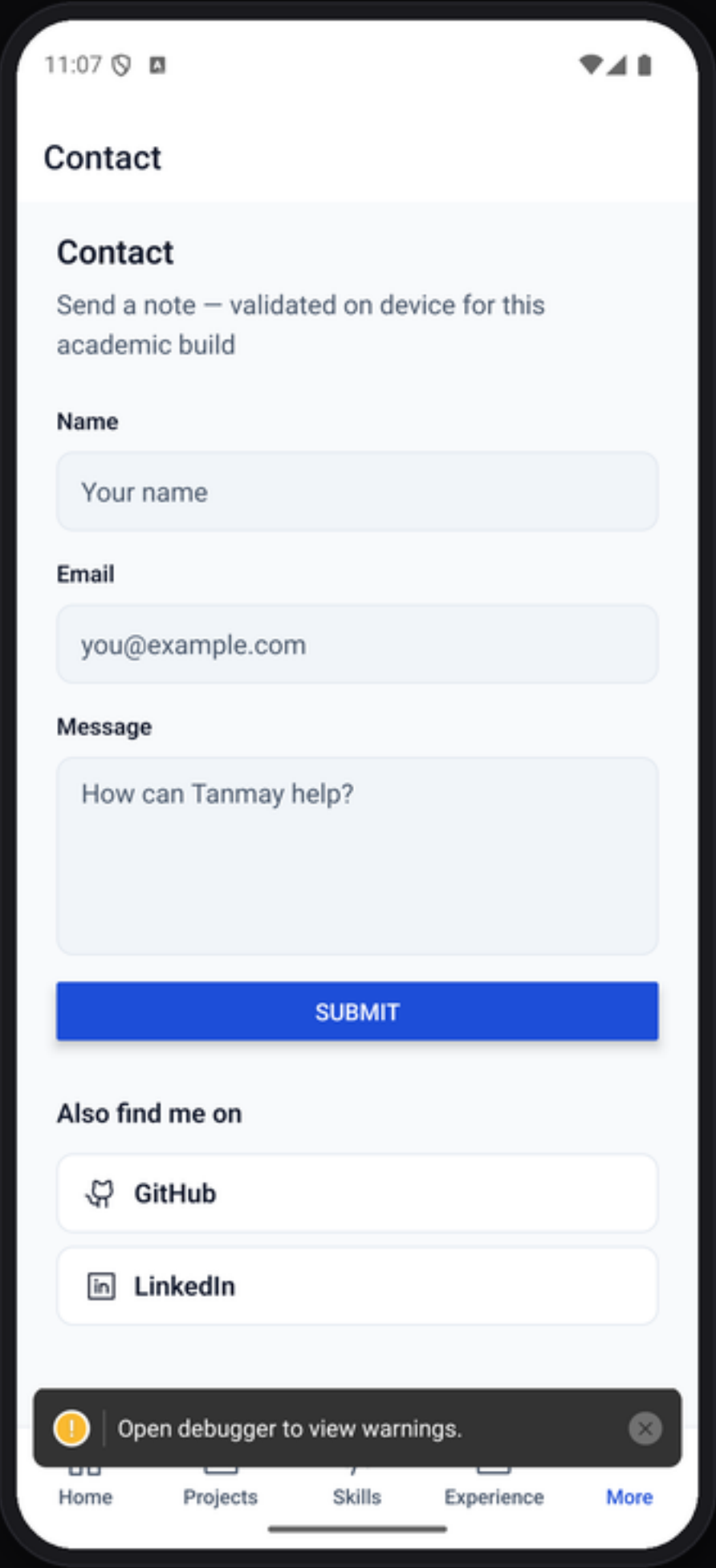
Horizontal row: timeline rail on the left, card content on the right.

Project card actions

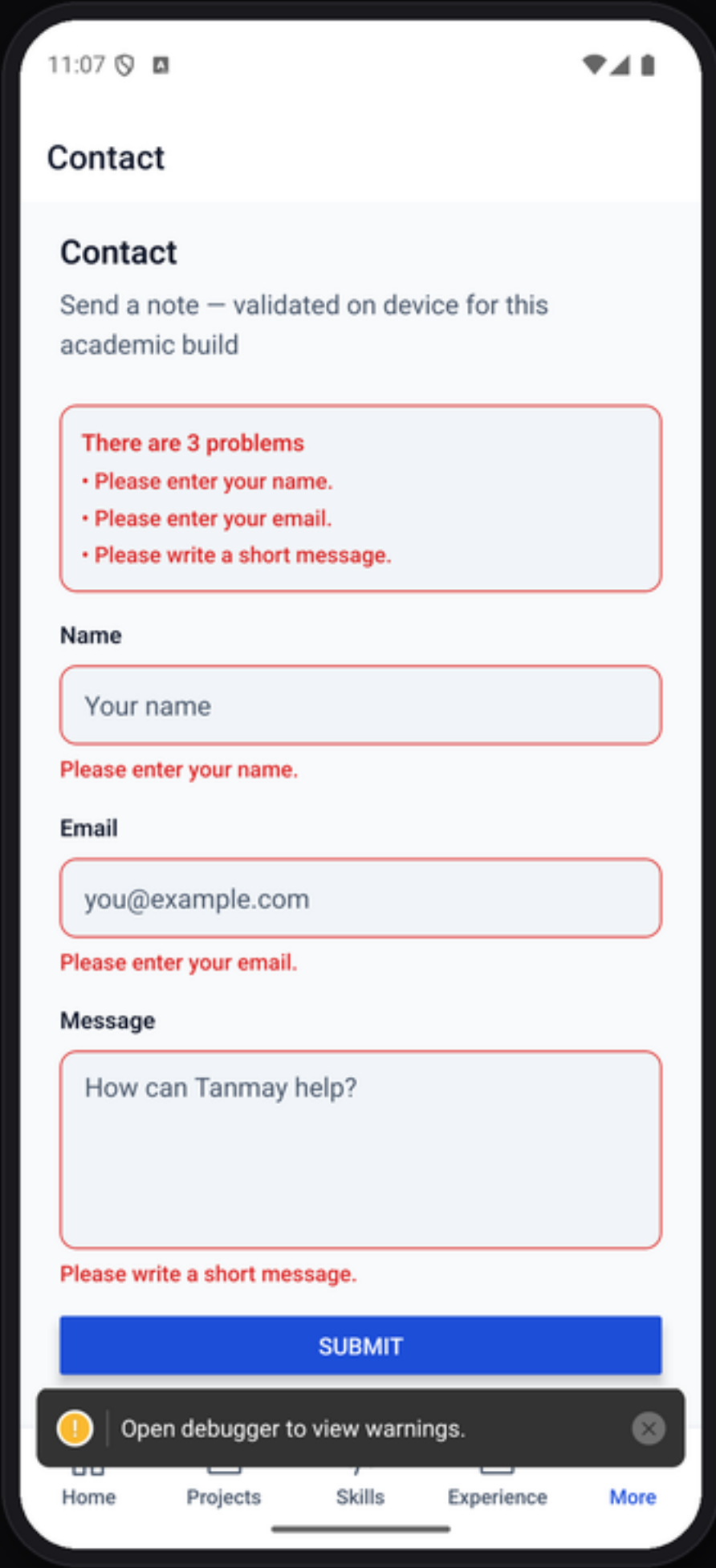
Details + GitHub sit in a Flexbox row inside ProjectCard.



Controlled inputs, then a press that means something



Empty form · TextInput



Submit · onPress errors

What the code is doing

Controlled TextInput

Name, email, message values live in `useState`. The field always shows state — never an uncontrolled buffer.

Button onPress

`ContactForm <Button onPress={handleSubmit} />`. Empty submit → error state. Valid submit → Alert.

Pressable CTAs

Home Explore buttons and ProjectCard GitHub/Details use Pressable for navigation and expand.

Validation UX

Red borders + summary box prove state changed. That is the assignment: input handling + a press handler.

Aimed at Excellent — and where to prove it



Functionality & Output

All sections run. Theme Switch. Form validation. External links. Pixel_9 install.



Code Structure & Syntax

src/ modular layout. TypeScript. Consistent component names. No one-file app.



Concept Implementation

Props, useState, eight core components, Flexbox, TextInput, onPress — all visible.



Readability & Documentation

File comments, docs/ reports, README, screenshot folders per assignment.

Show the app. Name the concept.

1	Home	Identity, photo, CTAs — stay on light theme first.
2	Dark mode	Toggle Switch at the bottom of Home. Say useState / isDark.
3	Projects	Props on ProjectCard. Expand Details.
4	Skills	Flexbox chips wrapping. SkillChip reuse.
5	Experience	AdMax + CodSoft from the resume. Rail + row.
6	Contact	Empty Submit → errors. Valid Submit → Alert.
7	Studio	Open android/. Mention Logcat + Metro.

QUESTIONS

Happy to walk any screen or any file.

Keep these two lines ready

1. Data lives in `src/data`; components stay reusable via `Props`.
2. `isDark` in `App.tsx` updates the whole theme when the Switch toggles.



Tanmay Shinde · TanmayPortfolio

React Native Community CLI · Pixel_9 · Assignments 1–6